

Correction-Aware Data Admission: Candidate-Bound Authorization and Field-Level Provenance for AI-Derived Updates

Liang Hanghao · Xuan Wentao ·
Chen Qile · Peng Peng

Abstract Human-reviewed AI-derived updates introduce a correction-specific authorization problem that is invisible in the final authoritative value. A machine proposal, a human-authorized value, and the exact reviewed candidate are distinct objects: even equal-valued candidates with equivalent retained review context can remain different authorization targets. We identify candidate-bound authorization as a distinct integrity requirement for corrected AI-derived updates when instance accountability is required. We specify a correction-aware admission relation connecting the exact candidate, its authorization, an explicit authorized value, a fresh authoritative predecessor, a complete successor, and total field-level sources. Safe/unsafe paired histories characterize five information classes whose omission can conceal different authority outcomes. An independently written exact-binding event journal and a relational reference realize this relation and agree on all 165 corresponding cases. A context journal accepts 15 equal-valued substitutions that both exact-binding mechanisms reject, separating content/context approval from instance authorization. A server-held review binding rejects nine request substitutions, while three display-only substitutions remain admissible in both paths, exposing the presentation boundary. The results establish an executable account of correction integrity across two persistence representations: preserve the proposal, commit the authorized value, and retain the exact reviewed instance and every field’s source.

Keywords data integrity · data provenance · human authorization · AI-assisted data acquisition · transactional admission · relational verification

1 Introduction

Intelligent information systems increasingly turn machine-produced information into persistent operational records through human review. Document intelligence and data curation illustrate a consequential case: a reviewer corrects a proposed value before authorizing its admission. Such a correction creates an authorization problem

Liang Hanghao · Xuan Wentao · Chen Qile · Peng Peng
College of Computer Science and Electronic Engineering, Hunan University, Lushan South Road, Yuelu District, Changsha 410082, China
E-mail: hnu16pp@hnu.edu.cn (Peng Peng, corresponding author)

that the final record cannot reveal. A correct final value does not establish which machine proposal was preserved, which value the human authorized, or whether the admitted candidate was the exact instance reviewed.

Consider a record whose quantity is 90. A recognizer proposes 100, and a reviewer authorizes 101. Admission must retain the machine proposal 100 while committing the human-authorized value 101; an unchanged batch-code field must retain its earlier source. The proposal explains the origin of the correction, whereas the authorization permits the committed value. Overwriting the proposal collapses these roles and can falsely attribute the corrected value to the machine. Correction requires the relation to permit

$$x_c \neq x_a. \quad (1)$$

A further distinction survives even when values and review context agree. Two persisted extraction attempts can have the same proposal, record, field, evidence, expected version, and retained review context, yet remain different candidate instances. Reviewing one does not authorize the other under an instance-accountability policy. *Equal-valued candidate substitution* exposes this difference without relying on a changed result or context mismatch. Candidate identity then becomes part of the integrity condition for authoritative admission: it identifies the object to which authorization applies.

We call this requirement *candidate-bound authorization*. For corrected AI-derived updates, it binds the human’s explicit authorized value to the exact persisted candidate while preserving the original proposal. The resulting admission relation connects

$$\begin{aligned} \text{exact candidate} &\rightarrow \text{candidate-bound authorization} \\ &\rightarrow \text{explicit authorized value} \\ &\rightarrow \text{fresh authoritative predecessor} \\ &\rightarrow \text{complete successor and total field sources.} \end{aligned} \quad (2)$$

Changed fields acquire transitions carrying their authorization; unchanged fields retain their exact prior sources. These bindings make the correction accountable both at commit and through subsequent queries.

We make three contributions:

C1: A correction-specific integrity requirement and admission relation. We formalize candidate-bound authorization and connect proposal preservation, explicit human value authority, predecessor freshness, complete successors, and total field sources in an executable contract.

C2: A characterization of failure-distinguishing information. We identify five classes: candidate identity/context, proposal/authorized-value roles, freshness, successor completeness, and field-source attribution. Safe/unsafe paired histories show how omitting each class makes different authority outcomes observationally indistinguishable. Proposition 1 states the formal scope of this characterization.

C3: Cross-representation realization and a decisive policy separator. An independently written exact-binding event journal and the relational reference agree on all 165 corresponding cases. Fifteen equal-valued substitutions separate both from context approval. Review-session controls locate the request-to-review and presentation boundaries; operational workloads characterize the cost of enforcing and querying the relation.

The gap concerns this end-to-end relation. Transactions can commit a substituted candidate atomically; freshness checks can validate its predecessor; provenance can

record its effects; and approval can authorize matching content. None of those conditions, by itself, establishes that the admitted instance is the one reviewed. Existing mechanisms provide the means to enforce the relation, but its authorization target and correction semantics must be specified explicitly.

The central finding is an authorization distinction, not a preference for one storage design. Sections 2–6 position, specify, and check the relation; Sections 7–8 evaluate its policy separator, review boundary, and operational costs. Section 9 develops implications and trust boundaries. Figure 1 illustrates Correction and field lineage.

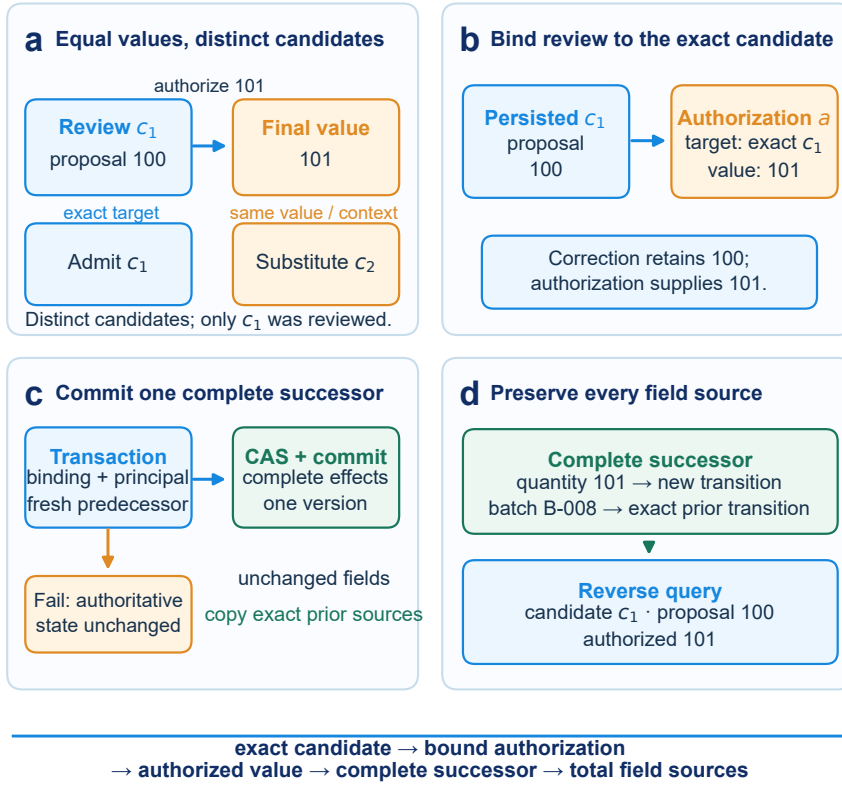


Fig. 1 The admission relation preserves both the reviewed instance and the original proposal through Correction. (a) Equal-valued candidates remain distinct authorization targets; (b) the authorized correction retains the proposal; (c) failed admission leaves authoritative state unchanged; (d) changed and unchanged fields retain their respective sources. Values illustrate a constructed example. Diagram preparation assisted by OpenAI Codex; image-generation prototype use is disclosed in Statements and Declarations

2 Related work

2.1 Transactions, update provenance, and event histories

Transactions provide atomic, durable state transformations [12], and optimistic concurrency control validates conflicting work before effects are installed [18]. These mechanisms support complete successors and commit-time predecessor checks. They do not determine the object of human authorization: an internally consistent transaction can admit a context-equivalent candidate that was never the reviewed instance.

Database provenance explains contributing data, derivations, and source locations [3, 6]. Curated-database provenance also records edits and copies across versions; Buneman et al. explicitly connect unchanged data to the preceding version and support source, history, and modification queries [4]. Reenactment reconstructs multi-version transaction provenance [1], while PROV-DM represents entities, activities, agents, derivation, and responsibility [21]. Exact source preservation and rich lineage are established capabilities. The question here is how they participate in an admission condition that preserves the machine proposal while assigning authority to a possibly different human-authorized value and the exact reviewed candidate.

Event sourcing reconstructs state from events [11]. An event journal can preserve proposals, corrections, reviews, versions, and field sources. Its persistence representation does not settle whether approval targets equivalent content or a particular instance. E1 holds the journal design and transactional mechanisms constant while adding the reviewed-candidate link. Agreement with the relational reference shows that candidate-bound authorization can be enforced in both representations; equal-valued substitution isolates the policy difference.

2.2 Human approval, correction, and data curation

Human-controlled integrity transitions have substantial precedents. Clark–Wilson combines well-formed transactions, authenticated execution, separation of duty, and reconstructible audit records [7]. Mershad et al. persist cell versions pending principal-investigator approval; selection by cell identifier and timestamp already supports version-specific content approval [20]. The present distinction concerns the authorization target when multiple persisted proposals share both values and retained review context.

Interactive cleaning separates system suggestions from human judgments. Guided Data Repair lets users confirm or reject repairs, retain a value, or supply a replacement [25]. Falcon converts corrections into candidate rules and recognizes that identical current effects can conceal different semantic validity [15]. Wrangler provides editable transformation histories, previews, and reusable scripts [17]; DBWiki combines identities, version history, atomic edits, and provenance queries [5]. These works establish correction and curation capabilities. We contribute an admission relation and its information characterization: preserve the original proposal, bind the explicitly authorized value to the exact reviewed instance, and carry both roles into a fresh complete successor with total field sources. The paired histories explain what disappears when these distinctions are omitted.

2.3 Governed AI state and persistent memory

Continuity Kernel is a close antecedent: it binds exact proposal identity, predecessor, pre-state authority, signed approver evidence, and lineage to an atomic complete accepted unit, with human preparation and multiple storage realizations [13]. Exact proposal binding is therefore an established neighboring principle. Our contribution specifies the correction-aware field relation and characterizes its failure-distinguishing information: proposed and authorized values retain separate roles, changed and unchanged fields retain exact sources, and equal-valued substitution isolates the required authorization target.

MemTxn separates admission, temporal visibility, and complete-state recovery [8]. MutMem binds signed retrieval-weight mutations to memory identity, provenance, signer epochs, and predecessor continuity [23]. LatticeMind manages claim status, conflict, supersession, and provenance [26]; Stored Is Not Supported governs assertion release through typed provenance from an authenticated accepted head [14]. These preprints address adjacent stages of persistent-state governance and may encode the relation studied here. Our claim concerns its explicit correction semantics, information characterization, and executable separation of content/context approval from instance authorization.

2.4 Binding and executable specification

DPoP binds possession proofs to request method and URI [10]; HTTP Message Signatures cover application-selected message components [2]. OWASP transaction-authorization guidance requires server-controlled approval data, invalidation after relevant changes, and a final gate [22]. E2 applies this binding discipline to transfer an exact candidate review into confirmation. Its request and display controls distinguish server-record consistency from faithful presentation to a reviewer.

Alloy and Kodkod support relational specification and bounded model finding [16, 24], while TestEra connects specifications to execution through concretization and abstraction [19]. We use these methods to examine the admission relation. The paired-history characterization provides the information distinction; the formal and persisted-state checks connect it to executable enforcement. Formal scope and implementation correspondence are stated in Sections 3, 4, and 6.

3 Authoritative-state admission contract

3.1 Candidate, authorization, and authoritative state

Correction-aware admission links an untrusted persisted candidate c , attributable authorization a , and authoritative pre-state Σ_v :

$$\text{Admit}(c, a, \Sigma_v) = \Sigma_{v+1}. \quad (3)$$

The candidate stays non-authoritative historical evidence. Proposal origin, value authority, and committed value remain distinct roles.

Let r be a record with a nonempty authoritative field set F_r . Its committed state Σ_v contains a total value map $V_v : F_r \rightarrow X$ and a total source map $S_v : F_r \rightarrow T$, where T is the set of durable fact transitions. A machine candidate for field f is

$$c = \langle r, f, e, v_{\text{exp}}, x_c, p, c_{\text{id}} \rangle, \quad (4)$$

where e is persisted evidence, v_{exp} is the expected current version, x_c is the machine proposal, and p identifies its producer. The identity c_{id} distinguishes exact persisted candidate instances even when proposal values and retained review context agree.

Evidence identity combines a content digest and canonical record/field locator:

$$EID(e) = \langle H(\text{bytes}(e)), L(r, f, u, \kappa) \rangle. \quad (5)$$

Here H is SHA-256, and L contains the owner record, field, normalized storage-relative URI u , locator version, and optional crop κ . Alloy abstracts content and locators as primitive domains. Identity uniqueness in its scopes relies on collision resistance and canonicalization/store integrity in the implementation (Online Resource 1, Section S2).

An attributable authorization is

$$a = \langle c_{\text{id}}, x_a, q \rangle, \quad (6)$$

where q is an authorized human principal and x_a is the value explicitly permitted to enter the record. The dual-value semantics are

$$\begin{aligned} \text{proposal}(c) &= x_c, & \text{authorize}(a) &= x_a, \\ \text{commit}(\Sigma_{v+1}, f) &= x_a, & \text{origin}(\Sigma_{v+1}, f) &= c. \end{aligned} \quad (7)$$

Accept has $x_c =_{\text{json}} x_a$. Correction has $x_c \neq_{\text{json}} x_a$; the candidate remains unchanged and the committed value derives its authority from a . Here origin identifies the retained proposal, not the authority for a corrected value.

Canonical value equality Write $x =_{\text{json}} y$ when sorted-key, whitespace-minimal JSON serializations agree. This distinguishes 2 from 2.0 and 1 from `true`, although Python equality does not. Change detection, value checks, tracing, the independent oracle, and projection use this relation. Section 6 reports the cross-layer repair.

3.2 History, observations, and outcomes

An authority-relevant history is

$$h = \Sigma_0 \xrightarrow{e_1} \Sigma_1 \xrightarrow{e_2} \dots \xrightarrow{e_n} \Sigma_n, \quad (8)$$

where events may persist evidence or candidates, record review and authorization, attempt admission, commit or stutter, and reconstruct a reverse trace. The abstraction describes semantic information rather than requiring particular table or column names.

Equal-valued substitution exposes one missing distinction. Extending the analysis across the admission relation yields five information classes for the modeled failures:

$$\mathcal{D} = \{D_C, D_V, D_F, D_B, D_S\}, \quad (9)$$

where D_C is candidate identity and context, D_V is dual-value attribution, D_F is freshness, D_B is batch/successor completeness, and D_S is total source attribution. For $I \subseteq \mathcal{D}$, $\pi_I(h)$ retains only the information classes in I . A certificate or stable handle may implement this identity, provided distinct persisted instances remain distinguishable despite equivalent values and retained context. Freshness may likewise use a version, state hash, epoch, or comparable discriminator.

To make “retains” precise, let $\text{obs}_d(h)$ be the observation function for class d and define

$$\pi_I(h) = \langle \text{obs}_d(h) \rangle_{d \in I}. \quad (10)$$

The functions are logical projections, not claims that a database must store five physically separate objects. A certificate can co-locate several classes. In a projection, however, a content address is treated as an opaque equality-preserving handle: its preimage is not available as a side channel. This prevents a nominally omitted value or version from being recovered merely because the concrete certificate hash covers it. The complete observation functions are given in Online Resource 1, Section S2.

The full-history authority-outcome label is

$$O(h) = \langle \text{status}, \text{successor}, \text{trace} \rangle, \quad (11)$$

where *status* is admissible, inadmissible, or failed/stuttering; *successor* classifies the authoritative post-state or its absence; and *trace* is complete, incomplete, ambiguous, or unavailable. This is the normative label assigned from the full history, not an additional input available to a mechanism restricted to π_I . A binary accept/reject label is insufficient because value-identical successors may differ in source completeness.

An observation set I fails to distinguish a failure family when there exist a safe history h_s and unsafe history h_u such that

$$O(h_s) \neq O(h_u) \quad \wedge \quad \pi_I(h_s) = \pi_I(h_u). \quad (12)$$

Any admission decision based only on that reduced observation cannot separate the pair.

The executable histories contain candidate and authorization registries, declared items, expected and observed versions, a connected commit sequence, endpoint maps, and transition sources. Projections and outcomes are derived from these relations. Schema fields determine total map domains; declared items determine required effects. Missing or conflicting sources remain failures in the domain. Online Resource 1, Section S2 gives the complete functions, finite tuple, and checker rules.

3.3 Conditional failure-distinguishability characterization

The five classes distinguish candidate substitution, false value attribution, stale replay, fragmented successors, and source ambiguity. Table 1 identifies a safe/unsafe pair for each class in the fixed observation basis.

Proposition 1 (conditional failure distinguishability) For each information class $d_i \in \mathcal{D}$, the declared failure model contains a safe history and a corresponding unsafe history whose authority outcomes differ but whose projections become equal when d_i is omitted. Consequently, an admission mechanism that observes only $\mathcal{D} \setminus \{d_i\}$ cannot distinguish the paired failure family under this model.

Table 1 Failure-distinguishing information classes in the declared observation model.

Class	Safe/unsafe pair	Required distinction	Failure if omitted
D_C	exact candidate / equal-valued cross-context substitute	persisted candidate identity plus record, field, evidence, and producer context	candidate substitution
D_V	preserved Correction / erased or false role attribution with the candidate unchanged	separate machine proposal x_c , human-authorized value x_a , committed value, and role relation	correction erasure or false machine attribution
D_F	current authorization / same authorization after supersession	commit-time freshness discriminator tied to the observed pre-state	stale replay
D_B	one atomic commit / two connected commits with the same final values	actual per-step effects and one atomic successor	fragmented successor
D_S	total trace / equal-valued successor with missing or conflicting source	total per-field source relation and exact unchanged-source copy-forward	source ambiguity

Proof sketch (by construction) For D_C , hold the value-role, temporal, batch, and source observations constant while changing non-temporal candidate context. For D_V , retain the same candidate, proposal value 100, and committed value 101, but falsely attribute the corrected value to the machine proposal. For D_F , change only the admission-entry freshness comparison. For D_B , compare one two-field commit with two connected single-field commits that reach the same final values and sources; the latter exposes an intermediate partial state. Effect domains and successor counts follow from these sequences. For D_S , remove one source anchor while retaining candidates, values, freshness, and the commit sequence. Each construction changes its selected observation coordinate and the derived outcome, while leaving the other four coordinates equal.

Thus, for every $d_i \in \mathcal{D}$, the constructed histories satisfy

$$\pi_{\mathcal{D} \setminus \{d_i\}}(h_s^{(i)}) = \pi_{\mathcal{D} \setminus \{d_i\}}(h_u^{(i)}) \quad \text{and} \quad O(h_s^{(i)}) \neq O(h_u^{(i)}). \quad (13)$$

Any decision function of the reduced projection receives the same input for both histories and cannot assign both distinct required outcomes. This establishes the claimed conditional irredundancy.

The executable checker validates each complete $\hat{h}$ and derives observations and outcomes from its relations; witnesses store neither. It checks domain validity, unequal outcomes, equal four-class projections, and unequal full projections. Schema fields define the total value/source domains, whereas declared item fields define the required effect domain.

The original D_C pair changes record context while retaining the candidate handle, establishing irredundancy of the combined class. An additional pair fixes the two-candidate registry and every context/value component, then selects either the authorized candidate or the equal-valued alternative. It isolates candidate binding

within this observation model. A single-field copy-forward control checks the distinction between schema and effect domains. Online Resource 1, Section S2 identifies the executable constructions and raw histories.

Scope of the result Proposition 1 establishes Eq. (13) for the declared failure, history, observation, and outcome model. Its scope is class-wise irredundancy, not universal minimality, schema uniqueness, or completeness over all admission failures. The checker validates the five class pairs, identity pair, and copy-forward control rather than arbitrary admissions. Separate Alloy checks examine the encoded assertions and conjunct-removal sensitivity in finite scopes.

3.4 Batch admission relation

A batch B is a nonempty set of items sharing one target record, principal, expected pre-version, and successor. Under Full, item fields are unique. Let

$$\Delta(V_v, V_{v+1}) = \{f \in F_r \mid V_v(f) \neq_{\text{json}} V_{v+1}(f)\}. \quad (14)$$

For a post-initial concrete admission,

$$\Delta(V_v, V_{v+1}) = \{i.f \mid i \in B\} = \{t.f \mid t \in T_{v+1}\}, \quad (15)$$

with one transition per item. At the first certificate-era version, the batch covers all of F_r , establishing total value and source domains. Deletion is outside the model.

For every $i \in B$, admission requires

$$\begin{aligned} i.r &= r, & i.f &\in F_r, \\ EID(i.e) &= EID(i.c.e), & i.v_{\text{exp}} &= v, \\ i.a.c_{\text{id}} &= i.c.c_{\text{id}}, & i.a.q &= q, \\ i.x &=_{\text{json}} i.a.x_a, & t_i.x &=_{\text{json}} V_{v+1}(i.f) =_{\text{json}} i.x. \end{aligned} \quad (16)$$

Together these instantiate the admission preconditions

$$\begin{aligned} &\text{Bind}(a, c) \wedge \text{Fresh}(c, \Sigma_v) \wedge \text{Authorized}(a) \\ &\wedge \text{Complete}(\Sigma_{v+1}) \wedge \text{Attributed}(\Sigma_{v+1}). \end{aligned} \quad (17)$$

The successor is atomic: all admission-time decision, authorization, transition, version, source-map, and audit effects commit together, or authoritative state stutters. This does not require rollback of evidence, candidates, or review intent persisted before the admission attempt. For a changed field, $S_{v+1}(f) = t_i$. For an unchanged field, value and exact source are copied forward:

$$f \notin \Delta(V_v, V_{v+1}) \Rightarrow V_{v+1}(f) =_{\text{json}} V_v(f) \wedge S_{v+1}(f) = S_v(f). \quad (18)$$

Alloy permits same-value admission for correspondence with the historical singleton model; concrete post-initial admissions require a value change. For submitted items U , the admitted batch is $B = \{i \in U \mid i.x \neq_{\text{json}} V_v(i.f)\}$, with $B \neq \emptyset$. A mixed submission admits exactly B and copies unchanged items' exact prior sources into the complete successor, creating no transition or authorization binding for $U \setminus B$. Thus Eq. (15) concerns admitted effects, not all submitted reviews. Recording renewed review or re-attribution without a value change requires a separate review-event path outside this contract; persisted candidate identity allows that path to distinguish equal-valued proposals.

3.5 Operational properties and trust boundary

Table 2 Admission properties and their operational interpretation.

Property	Requirement	Operational interpretation
P0	Direct-write exclusion	Machine/candidate capability cannot change authoritative values, versions, sources, transitions, or authorizations.
P1	Candidate non-substitutability	Authority refers to the same canonical persisted certificate, not merely an equal value or analogous candidate.
P2	Context-bound admission	Record, field, persisted evidence content and locator, and expected version agree across the item and certificate.
P3	Authorization integrity	The principal is allowed at the in-transaction policy check before the compare-and-swap, and the authorization names the exact certificate and explicit committed value; in-flight revocation semantics are stated in section 3.5.
P4	Freshness	The expected version equals the transactionally observed current version; stale or competing decisions fail closed.
P5	Successor integrity	One atomic successor has an item–transition bijection, exact changed-field effects, and complete changed/unchanged framing.
P6	Trace soundness	Each committed field resolves through its exact source transition to consistent authorization, certificate, evidence, producer, and version relations.

Model output and candidate-only callers are untrusted. The trusted base comprises the admission service, adapter, exercised transaction semantics, canonicalization and hashing, and principal policy. Host authentication, session integrity, and faithful review presentation are external assumptions. The confirmation API constructs its binding from caller-supplied candidate and value; the contract preserves that binding, not factual truth or protection from a compromised host.

Authorization timing and revocation The in-transaction principal check precedes compare-and-swap and authority writes. Revocation visible before the check causes zero-write rejection; a later overlapping revocation does not cancel an in-flight commit. This is commit-entry revalidation. Online Resource 1, Section S2 states the policy-lock and stronger-revocation boundaries.

4 Bounded relational characterization

Relational analysis examines the contract’s legal steps, preservation properties, and sensitivity to removing individual bindings. It complements the observation-level construction by making the admission predicates executable.

The Alloy model represents records, fields, ordered versions, evidence, candidates, certificates, principals, authorizations, transitions, and states. Once a record is

authoritative, its value and source maps cover exactly its field domain. **MachineEvent** can extend candidate/evidence sets while framing authority (P0). **BatchAdmissionEvent** has a nonempty item set in one envelope. **TamperEvent** freezes values and source pointers while modifying transitions.

The base effect advances one version and applies item values and sources. Full adds unique fields, complete snapshot shape, preexisting certificate/evidence, contextual and authorization bindings, freshness, and an exact item–transition bijection. Separating these conjuncts permits controlled removal. Each ablation uses at least two items: one violates only the selected conjunct and another remains Full-valid. The reduced relation admits that set; Full rejects it with authoritative-state stutter.

Six legal commands exercise Accept, Correction, mixed batches, same-value admission, initial coverage, and singleton admission. Eleven ablations test encoded bindings. Three source attacks remove, replace, or duplicate an anchor after a legal prefix. Fourteen assertions per profile check framing, preservation, singleton correspondence, and trace conditions. The class-to-conjunct map and complete command descriptions are in Online Resource 1, Section S2.

The positive trace assertion requires a trace-complete predecessor and no stray successor transition. It checks admitted fields for one encoded step. Reachable initial and carry-forward witnesses establish a nonempty premise; removing post-state certificate binding changes UNSAT to SAT. Concrete tracing additionally validates predecessor prefixes. This distinguishes bounded preservation from the observation-level proposition.

Two scope profiles increase representative bounds from two records, three fields, six versions, and four states/events to three records, four fields, eight versions, and five states/events. Individual domain bounds remain fixed by deposited commands. SAT denotes a found witness; UNSAT denotes no counterexample within that command’s finite scope [16, 24].

Table 3 Bounded Alloy outcomes in the two declared scopes. SAT denotes a found witness; UNSAT denotes no instance within the encoded scope. Trace SAT counts witnesses to the antecedent of the conditional positive trace assertion.

Profile	Legal SAT	Ablation SAT	Attack SAT	Trace SAT	Total SAT	Total UNSAT	Commands
S1	6	11	3	2	22	14	36
S2	6	11	3	2	22	14	36

5 Transactional realization

5.1 Persistent objects and admission

AUTO-DECTE enforces the admission relation in Python, SQLAlchemy, and SQLite. **CandidateWritePort** appends candidates/evidence, **AuthorityReadPort** loads persisted objects, and **FactAdmissionPort** admits a complete bundle. Recognition receives the candidate facade; trusted review receives admission. These process capabilities implement P0 at the service boundary.

Here c_{id} abstracts `certificate_id`, the hash of the full canonical certificate, including the persisted instance identifier `candidate_id`, creation time, and proposal. The certificate also retains the target, producer/version, selection artifact, expected fact version, evidence hash, and canonical locator. A decision names reviewer and candidate; its immutable binding fixes the exact certificate and authorized value. Transitions preserve this chain for Accept and Correction. The revocable principal-role mapping is checked inside the transaction.

Admission reloads evidence and reconstructs its canonical locator, checking ownership, content hash, and record/field location against the certificate. The adapter repeats authoritative preflight checks transactionally:

1. reload the field domain, current version, complete predecessor, candidates, evidence, and principal policy;
2. derive initial coverage or later changed fields using canonical JSON equality, rejecting duplicate, hidden, empty, or incomplete effects;
3. validate unused identities and the decision–binding–certificate–evidence–transition relation, including agreement of authorized, transition, and successor values;
4. compare-and-swap the expected current version;
5. create one transition per admitted field, copy exact unchanged sources, and commit decision, binding, complete version, projections, and audit together.

CAS is the first authority write. Failure before commit rolls back admission. Unique indexes and foreign keys reinforce bindings and transition uniqueness. The zero-write baseline is immediately before admission, after any candidate or review preparation. Connection parameters and locator rules are in Online Resource 1, Section S1 and the artifact.

The confirmation API receives candidate, value, and principal from its caller and constructs the binding at confirmation. E2 evaluates an earlier server-held review binding. Manual-source compatibility derives a certificate at confirmation and is outside the preexisting-machine-candidate claim. The historical restricted agent interface permits proposal and verification, with confirmation retained by the host (Online Resource 1, Section S5).

5.2 Reverse trace and source evolution

Tracing checks each field in every prefix version. Initial and changed fields need their unique transition; unchanged fields retain the exact prior source. The verifier checks source record/field/version, committed value, decision, authorization, certificate, producer, evidence hash/locator, and expected-version relations.

A trace is **complete** when these relations agree, otherwise **incomplete** with reasons. It does not repair or substitute sources; legacy pre-certificate states are identified separately. Snapshot completeness covers every field’s value and source; trace completeness checks persisted authorization lineage under the trust boundary in section 3.

6 Formal–concrete projection and executable conformance

The conformance layer asks whether persisted candidate, authorization, successor, and source relations satisfy the formal admission predicate. A test-side abstraction

connects selected persisted events to the model:

$$\alpha : \text{PersistedPrePost} \rightarrow \text{BatchAlloyInstance}. \quad (19)$$

It independently reads targets, field domains, evidence/locators, certificates, decision/binding rows, version values/sources, transition sets, and pre/post identity sets. Canonical JSON classes become Value atoms, certificates become primitive identities, and a decision/binding becomes Authorization. The complete mapping is in Online Resource 1, Section S2.

A committed wrapper fixes the instance and checks `legalAdmission[event, Full]`. A rejected wrapper requires equal full-database digests, state stutter, and failure of Full. Twenty single-dimension mapping mutants exercise context, freshness, authorization, preexistence, transition, and coverage sensitivity. The adapter shares neither the production planner nor the trace builder.

A second oracle reads all SQLite application tables and checks domains, source evolution, transitions, version anchors, lineage, and values. Five relation mutants exercise its sensitivity. The 35-case catalogue includes five legal cases and thirty rejection, corruption, schema-prevention, stateful-evolution, or oracle-mutant cases. Rejections require the expected public error and unchanged deterministic database digest; post-persistence corruption instead requires a changed digest and trace/oracle diagnosis.

Thirteen regressions test canonical equality, including unchanged controls, numeric/Boolean and nested changes, and no-op rejection. Seven fail before the equality repair and all pass afterward. A separate witness-domain repair retains the five class pairs and adds identity and copy-forward controls (Online Resource 1, Section S2).

Nine intended projections cover singleton and multi-field Accept and Correction, mixed batches, initial coverage, copy-forward, staleness, and whole-batch rejection. They provide selected bounded correspondence. Three scope differences remain explicit: concrete post-initial no-ops are rejected; SQLite uniqueness prevents one modeled duplicate-source state; and concrete prefix tracing checks corruptions and predecessor relations beyond the formal single-level anchor predicate. Thus the positive Alloy assertion concerns one encoded admission step under its well-formed-pre-state condition.

7 Evaluation protocol

7.1 Questions and evidence layers

E1–E3 provide controlled implementation evidence through constructed histories, scripted review interactions, and fixed workloads. E1 tests why exact candidate binding matters when values and retained context agree. E2 locates the review-to-confirmation boundary of that guarantee. E3 measures the costs of enforcement and querying.

RQ1 Do the formal and concrete checks support the admission contract, and what changes when a rich event journal records the exact reviewed candidate?

RQ2 Which inconsistencies between recorded review intent and confirmation can server-held session binding reject, and which remain outside its guarantee?

RQ3 What admission, reverse-trace, and storage costs occur on the repaired implementation under the declared workloads?

E1–E3 use the equality-repaired reference at commit `c6d512843c905cab6d8521dd8c914f7fb26d85ae`, after rerunning implementation regressions and harness checks. The earlier Alloy, projection, and catalogue results remain separate evidence. The prospective local protocol, pilot amendments, environment, raw receipts, and execution status are documented in Online Resource 1, Sections S1 and S6. E1–E3 make no hosted-model calls; historical agent runs remain separate in S5.

Execution, query-answer, browser-case, and timing counts have separate denominators.

Figure 2 summarizes the evidence route from the admission relation to E1–E3.

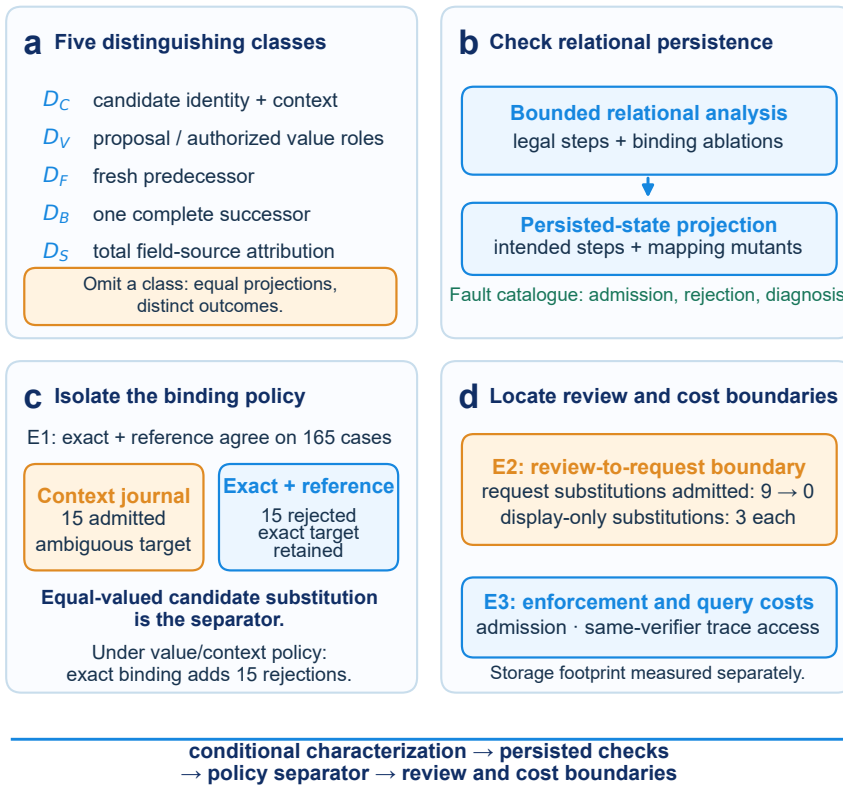


Fig. 2 Evidence route from the conditional characterization to persistence checks and E1–E3. E1 supplies the policy separator; E2 tests request binding and the presentation boundary; E3 characterizes enforcement and query costs. Displayed E1/E2 results are read from archived tables; timing totals are checked against CSV summaries. Diagram scripting assisted by OpenAI Codex

7.2 RQ1: rich-context and exact-binding comparison

A separately written `sqlite3` journal retains immutable candidates/events, review context, versions, transactions, and complete field sources. Its *context* configuration checks field, version, evidence, and retained context while omitting candidate/certificate identifiers and instance-creation time. Its *exact* configuration adds the reviewed-candidate link. The third arm invokes the reference’s actual transaction. We score both instance authorization and the looser value/context policy.

Fifteen inputs cross eleven case families and three mechanisms. Twelve deterministically selected archived proposal cells contain four distinct values; three synthetic type controls bring the total to seven. They enter newly constructed histories, rather than complete historical-task replays. Cases exercise Accept, Correction, multi-field admission, equal-valued candidate and different-evidence substitutions, staleness, replay, unauthorized value, and three interrupted transaction locations. Online Resource 1, Section S6 lists the inputs and cases.

After acceptance, an independent reader checks six provenance questions on three fields: proposal, authorized value, reviewer, reviewed candidate, current value, and source version. Each accepted history contributes eighteen answers, classified as correct, wrong, ambiguous, or unavailable. Rejection requires the expected reason and unchanged logical fact-database digest, measured after authorization preparation. Prior review records may persist. Original strengthened-baseline continuity checks have a separate denominator.

7.3 RQ2: review-to-confirmation boundary

Local Chrome/Playwright runs an instrumented review fixture backed by `ReviewForms.confirm`. An independent driver records the displayed candidate, proposal, and authorized value before confirmation. Three JSON inputs cross ten interactions and two paths, giving sixty cases; six confirmations separately prepare replay setups. Positive controls cover acceptance, correction, and reselection. Candidate/value/principal request substitutions, stale/replayed requests, interrupted commits, and display-only substitution probe distinct boundaries.

The original path is compared with a server-held session binding candidate, version, reviewer, and value. Acceptance, agreement with recorded review intent, and writes on rejection are recorded. The fixture uses fixed principal labels and a separate session database. The resulting request and presentation boundaries are interpreted in section 9.

7.4 RQ3: current-version costs

Two ten-cell admission comparisons contribute 4,000 observations each; a 36-cell trace comparison contributes 14,400. Each cell has five warmups and 200 timed calls per arm in seeded randomized order, for 22,400 timed observations excluding warmups. Environment versions and full grids are in Online Resource 1, Sections S1, S4, and S6.

The mechanism comparison times complete confirmation on shared recognition-source workloads. Construction and database cloning are excluded: the journal

opens its connection before timing, whereas the reference may connect lazily inside confirmation. The study measures these complete call boundaries; section 9 explains their implementation differences.

A separate manual-source materialization ablation times persistence of a prevalidated delta while retaining authority rows, complete sources, transactionality, and CAS. Its result must match the precomputed relational fingerprint. Validation and planning occur outside that timer; its full-path column belongs to a different workload from the mechanism comparison.

The trace comparison holds the verifier fixed and changes three bulk repository reads to identifier enumeration and point lookups. Every returned complete object must agree. The grid varies record width, history length, and total records. Storage fixtures populate full/reduced schemas at three transition scales; their footprint excludes external evidence/model outputs, and the reduced schema has fewer audit obligations.

8 Results

8.1 RQ1: exact binding and persisted conformance

E1: equal-valued candidate separation Fifteen equal-valued candidate substitutions separated the authorization policies. The context journal accepted all 15: their values and retained context satisfied its policy, although the admitted instances differed from the reviewed targets. Both exact-binding mechanisms rejected them. The exact journal and relational reference agreed on all 165 paired decisions and query-answer sets. All mechanisms rejected substitutions with different retained evidence.

The separator concerns the required authorization target. Under instance accountability, the rejected substitutions are unauthorized; under the looser value/context policy, the exact mechanisms make 15 additional rejections. Thus the experiment isolates a policy choice rather than treating context approval as inherently incorrect.

All 495 executions completed without harness errors. Each exact mechanism accepted 45 legal cases and rejected 120 without fact-database writes after authorization preparation, including all three rollback locations.

Accepted context-journal histories produced 1,020 correct answers and sixty ambiguous reviewed-candidate answers. Each exact mechanism produced 810 correct answers and none wrong, ambiguous, or unavailable. Totals reflect eighteen answers per accepted history, with more acceptances under the context policy. The ambiguous answers expose the same instance distinction after commit.

Table 4 E1: 165 executions per mechanism. Instance violations are accepted cases violating the instance policy. Extra rejections are rejected cases valid under the value/context policy. Query counts refer to field-level answers after acceptance. All arms had zero value/context-policy violations and zero rejections with fact writes.

Mechanism	Accepted cases	Instance violations	Extra rejections	Query answers	
				Correct	Ambiguous
Context journal	60	15	0	1,020	60
Exact journal	45	0	15	810	0
Reference	45	0	15	810	0

The original strengthened-baseline continuity run reproduced seven-of-eight single-history agreement, with equal-valued substitution as the separator; its paired-history distinction is a separate ninth check. The richer exact journal closes that gap in E1 by preserving the authorization link. The evaluated relation is therefore enforceable in both storage designs.

Formal and persisted support All 72 Alloy outcomes matched the manifests, with legal, ablation, source-attack, and nonempty-precondition witnesses in both profiles. The checked assertions had no counterexamples within scope. The intended projections were satisfiable, and the mapping mutants were unsatisfiable. Every catalogue case, both stateful profiles, and all 33 rollback/concurrency tests passed in the retained execution.

The illustrative lifecycle retained proposal 100, separately authorized correction 101, exact sources for the other two fields, and a complete trace/export. Six archived AI-origin lifecycles and ten pinned-interface controls also matched their outcomes. The new execution passed 85 selected regressions and ten harness checks. These are separate evidence layers; the retained tests were not all rerun for E1–E3.

8.2 RQ2: request binding and display controls

The server-held gate rejected all nine request substitutions that the original path accepted despite disagreement with recorded review intent. Both paths accepted three display-only substitutions. Request binding therefore separates request-to-review consistency from presentation integrity.

Each path completed thirty browser cases and accepted all nine legitimate controls. Both rejected stale submissions, replays, and precommit interruptions without fact writes. In the display-only controls, the page showed B while server and request continued with A. The original transaction preserves the caller-supplied binding; the gate additionally checks it against the earlier server-held review record.

8.3 RQ3: admission, trace, and storage costs

Admission Both admission comparisons completed ten cells. Shared mechanism queries agreed; materialized states matched their precomputed fingerprints. Table 6 shows representative cells; Online Resource 1, Section S4 retains every cell.

Table 5 E2: accepted cases by interaction family, with three JSON values per named interaction. Each path has thirty cases; replay setup is additional. All nine original-path rejections and eighteen gate rejections made zero fact writes.

Interaction family	Cases/path	Original	Session gate
Accept, Correction, reselection	9	9	9
Candidate, value, principal request substitutions	9	9	0
Stale, replay, precommit interruption	9	0	0
DOM-only substitution	3	3	3
All cases	30	21	12

Table 6 Representative E3 admission cells: p50/p95 (ms), 200 observations per arm/cell. Recognition and manual-source columns use separate workloads; prevalidated timing excludes validation and planning. All ten cells are in Online Resource 1, Section S4.

Fields	Changed	Recognition-source mechanisms		Manual-source ablation	
		Reference	Exact journal	Full	Prevalidated
1	1	28.79/50.50	10.40/13.32	23.23/33.76	9.97/14.63
32	32	187.24/292.72	13.07/17.77	122.68/200.00	13.89/22.61
128	128	712.00/1039.83	22.09/30.53	427.59/621.20	23.50/37.43

With 128 fields all changed, median reference confirmation was 712.00 ms and exact-journal confirmation 22.09 ms. Both enforce exact binding; these are complete-call costs for the compared implementations. In the separate manual-source ablation, full confirmation was 427.59 ms and prevalidated materialization 23.50 ms. The latter times persistence after validation and planning, as defined in section 7.

Reverse trace All 36 paired cells completed 14,400 calls with identical full returned objects. At 128 fields, 100 versions, and 1,000 records, bulk p50/p95 was 170.24/216.75 ms versus 509.09/595.85 ms for point access, a median ratio of 2.99.

Bulk tracing used twelve SQL statements in every measured cell; the largest point-access cell used 693. With the same current verifier in both arms, the result attributes the measured difference to repository access. The constant statement count concerns round trips; field and history processing still contribute to total work.

Schema footprint Full-schema sizes were 1.52, 13.16, and 134.53 MiB at the tested transition scales, with roughly 1.1 kB incremental main-database storage per transition relative to the reduced fixture.

These are database footprints for the full and reduced audit obligations defined in section 7.

Table 7 E3 trace: representative cells with 128 fields and 1,000 total records. Latencies are p50/p95 milliseconds; each arm has 200 timed observations per row. SQL counts are constant within each row and arm. The complete 36-cell grid is deposited.

Versions	Latency (ms)		SQL statements	
	Bulk	Point reads	Bulk	Point reads
1	28.68/43.26	209.81/256.03	12	396
10	39.69/48.16	233.82/285.43	12	423
100	170.24/216.75	509.09/595.85	12	693

Table 8 E3 synthetic schema footprint after checkpointing. Incremental bytes per transition equal the full-minus-reduced main-database size divided by transition count. External evidence files are excluded.

Transitions	Full (MiB)	Reduced (MiB)	Incremental bytes/transition
1,000	1.52	0.48	1085.4
10,000	13.16	2.88	1077.2
100,000	134.53	28.45	1112.3

8.4 Separate historical integration

The archived matrix retains 1,260 planned runs. Strict benign yield was 320/360 (88.89%) and post-hoc endpoint yield 331/360 (91.94%), with 25 unscorable runs; all 93 runtime failures remain recorded. Fixed host checks recorded no unauthorized mutation in 720 invalid-tuple calls or 179 capability-unavailable branches, the latter making no admission call. This reporting separates task completion from authority checks, analogous to AgentDojo’s task/security outcomes [9]. The historical evidence supports restricted-interface integration separately from E1/E2; Online Resource 1, Section S5 preserves conditional denominators, configuration-specific failures, and annotation.

9 Discussion and limitations

9.1 What the results establish

The findings identify candidate-bound authorization as a distinct integrity requirement for human-corrected AI updates when instance accountability is required. Correction separates the machine proposal from the value permitted to enter authority; candidate substitution separates content/context equivalence from identity of the authorization target. An accountable update must therefore preserve three objects together: what the machine proposed, what the human authorized, and the exact candidate reviewed. Total field sources carry these distinctions into the successor and make them queryable after commit.

Equal-valued candidate substitution is the decisive policy separator. The context journal admits all fifteen substitutions because their values and retained context match its approval policy; both exact-binding mechanisms reject them because they target different instances. The sixty ambiguous reviewed-candidate query answers expose the same distinction after admission. The experiment establishes a difference in accountability granularity: content/context approval permits interchangeability that instance authorization forbids.

The relation is executable across two persistence representations. The independently written exact-binding journal and relational reference agree on all 165 corresponding decisions and query-answer sets. Their agreement shows that enforcing the relation does not depend on the reference schema. The five paired histories supply complementary knowledge: retaining the final value cannot replace candidate, role, freshness, completeness, or source distinctions when an omitted class conceals a different authority outcome.

E2 identifies where the binding must be established. A server-held review record rejects the nine request substitutions that the original confirmation path admits, protecting the transfer from recorded review to request. Both paths still admit three display-only substitutions. Consistency with a server record therefore leaves faithful display as a separate requirement. E3 supplies operational cost characterization for admission, trace access, and storage. These measurements inform implementation choices; the contribution is the authorization distinction, executable admission relation, and evidence for its enforcement and boundary.

9.2 Trust and validity boundaries

The server-held gate establishes request-to-review consistency for the tested substitutions. Display-only controls identify the remaining presentation boundary. Host authentication, review-channel integrity, principal policy, hashing/canonicalization, and database execution remain trusted. Lineage establishes recorded authorization and sources; factual truth requires independent validation. Evidence checks compare stored hashes and locators rather than re-reading external bytes. Detecting external replacement therefore requires immutable storage or additional integrity checks. Commit-entry policy revalidation also leaves later-revoked in-flight operations able to commit (Online Resource 1, Section S2).

The model conditions and class-wise scope of Proposition 1 are specified in section 3. Alloy checks finite scopes; section 6 states the selected-projection boundary and the formal no-op, schema-uniqueness, and trace-prefix differences. Distributed revocation, other database engines, deletion/redaction, and retention expiry require additional design and evidence.

E1 reuses seven distinct proposal values in constructed cases, and the journals are same-study implementations. E2 uses a local scripted fixture, fixed principals, and separate session/fact databases. Human attention, hostile hosts, and cross-database crash atomicity require separate study. Historical hosted-agent outcomes and author-involved annotation have distinct scoring, missingness, and failure records in Online Resource 1, Section S5.

E3 characterizes one machine with uncontrolled background activity. The admission arms differ in schema, ORM, validation, functionality, and connection lifecycle. Their measured costs therefore do not isolate exact-binding overhead. The separate materialization ablation excludes validation/planning and is not a complete admission path. The paired trace study holds the verifier fixed; its constant SQL count leaves processing, prefix traversal, and memory free to grow with history and fields. Storage fixtures exclude external evidence/model outputs and compare different audit obligations. Online Resource 1, Sections S4, S6, and S7 retain the complete workload and validity details.

9.3 Applying the relation

Adoption begins with the approval policy and one correction traced through persisted records. Identify the reviewed candidate, authorized value, predecessor check, and changed/unchanged source links. Retain a legal control and a failure probe for each boundary. Online Resource 1, Section S3 provides the worked example and inspection checklist.

The relation can be enforced in relational tables or event journals by preserving these objects and validating their bindings together. Concrete post-initial admission creates transitions for changed values; renewed review or re-attribution without a value change belongs to a separate review-event path. An independently implemented deployment with operational reviewers would provide the next test of transferability.

10 Conclusion

Human-reviewed AI corrections introduce an authorization problem that cannot be resolved from the final authoritative value. The machine proposal, human-authorized value, and exact reviewed candidate retain distinct roles. Equal-valued candidates with equivalent retained review context can remain different authorization targets; when instance accountability is required, candidate identity participates in admission integrity.

We formalize candidate-bound authorization in a correction-aware relation linking the reviewed instance and explicit authorized value to a fresh predecessor, a complete successor, and total field sources. Five safe/unsafe paired histories characterize the information needed to distinguish the specified failure families. The independently written exact-binding event journal and relational reference agree on all 165 corresponding cases, while fifteen equal-valued substitutions distinguish them from context approval.

Server-held binding protects review-to-request consistency in the nine request-substitution cases; the three display-only controls identify presentation integrity as a separate boundary. Operational measurements characterize enforcement and provenance costs. The resulting contribution to intelligent information systems is an executable account of accountable correction: preserve the proposal, commit the authorized value, bind authority to the reviewed instance, and retain every field’s source.

Statements and Declarations

Data and software availability

The study uses synthetic and public lifecycle inputs. The new comparison, browser-boundary, and current-version cost evidence is deposited with code, raw receipts, database snapshots, screenshots, dependency locks, and manifests at the frozen experiment archive. The reference implementation is fixed at commit `c6d512843c905cab6d8521dd8c914f7fb26d85ae`, under `latest/code/implementation-fixed`. Its formal models, observation witnesses, and earlier annotation and integration evidence are accessible from the frozen formal and historical evidence archive. Historical v8 evidence remains separately versioned under commit `abfdb817835a2c0953e5a26d854f34d99347dc6a`. Online Resource 1, Section S1 maps the evidence layers and describes restoration of the losslessly compressed large database. Inspecting the deposit and reproducing the three new experiments requires no paid model calls; repeating historical hosted-agent runs requires provider credentials.

Ethics approval and consent to participate

The new experiments use constructed data and scripted browser interactions, with no new human participants. In the historical annotation study, one author (X.W.) and one non-author volunteer independently coded benign completion outcomes; L.H. adjudicated the inter-human disagreement and separately reviewed eleven human-rule disagreements against the strict-trajectory criterion. X.W. also completed the

textual-acknowledgment annotation. Only pseudonymous annotator identifiers were collected; both benign-endpoint coders were informed of the task and data use, and the non-author coder participated voluntarily. Annotation evaluated generated outputs rather than annotator characteristics. The authors assessed this activity as not requiring ethics approval. No animal subjects were involved.

CRediT authorship contribution statement

Liang Hanghao: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Validation, Visualization, Project administration, Writing—original draft, and Writing—review & editing.

Xuan Wentao: Methodology, Formal analysis, Investigation, Data curation, Validation, Visualization, and Writing—review & editing.

Chen Qile: Investigation and Validation.

Peng Peng: Supervision and Writing—review & editing.

Funding

This research received no external funding.

Competing interests

The authors declare no competing interests.

AI-assisted preparation

The authors used DeepSeek, OpenAI Codex, and Claude for code, analysis checks, literature organization, and manuscript and diagram preparation. OpenAI image generation supplied a design prototype for Figure 1; its model/version was not recorded. The final diagrams were rendered from scripts using Matplotlib 3.10.8. OpenAI Codex also assisted with contribution-focused restructuring of the Abstract, Introduction, Related Work, Discussion, and Conclusion. The authors verified sources, results, and figure content and take responsibility for the article.

Supplementary information

Online Resource 1 contains the artifact guide, formal observation and projection details, worked correction example, full workload grids, and historical integration and validity records.

References

1. Arab BS, Gawlick D, Krishnaswamy V, et al (2018) Using reenactment to retroactively capture provenance for transactions. *IEEE Transactions on Knowledge and Data Engineering* 30(3):599–612. <https://doi.org/10.1109/TKDE.2017.2769056>
2. Backman A, Richer J, Sporny M (2024) Http message signatures. RFC 9421, Internet Engineering Task Force, <https://doi.org/10.17487/rfc9421>, URL <https://www.rfc-editor.org/rfc/rfc9421.html>
3. Buneman P, Khanna S, Tan WC (2001) Why and where: A characterization of data provenance. In: *Database Theory—ICDT 2001, Lecture Notes in Computer Science*, vol 1973. Springer Berlin Heidelberg, p 316–330, https://doi.org/10.1007/3-540-44503-X_20
4. Buneman P, Chapman A, Cheney J (2006) Provenance management in curated databases. In: *Proceedings of the 2006 ACM SIGMOD international conference on Management of data*, pp 539–550, <https://doi.org/10.1145/1142473.1142534>
5. Buneman P, Cheney J, Lindley S, et al (2011) The database wiki project: A general-purpose platform for data curation and collaboration. *ACM SIGMOD Record* 40(3):15–20. <https://doi.org/10.1145/2070736.2070740>
6. Cheney J, Chiticariu L, Tan WC (2009) Provenance in databases: Why, how, and where. *Foundations and Trends in Databases* 1(4):379–474. <https://doi.org/10.1561/19000000006>
7. Clark DD, Wilson DR (1987) A comparison of commercial and military computer security policies. In: *1987 IEEE Symposium on Security and Privacy*, pp 184–194, <https://doi.org/10.1109/sp.1987.10001>
8. Cui H, Tang Z, Yao Z, et al (2026) MemTxn: A transaction boundary for source-supported updates and complete-state recovery in agent memory. <https://doi.org/10.48550/arXiv.2607.27834>, URL <https://arxiv.org/abs/2607.27834>, preprint, arXiv:2607.27834
9. Debenedetti E, Zhang J, Balunovic M, et al (2024) AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In: *Advances in Neural Information Processing Systems*, vol 37. Neural Information Processing Systems Foundation, pp 82895–82920, <https://doi.org/10.52202/079017-2636>
10. Fett D, Campbell B, Bradley J, et al (2023) OAuth 2.0 demonstrating proof of possession (DPoP). RFC 9449, Internet Engineering Task Force, <https://doi.org/10.17487/RFC9449>, URL <https://datatracker.ietf.org/doc/rfc9449/>
11. Fowler M (2005) Event sourcing. Architecture pattern article, URL <https://martinfowler.com/eaaDev/EventSourcing.html>, published 12 December 2005; accessed 26 September 2026
12. Gray J (1981) The transaction concept: Virtues and limitations. In: *Proceedings of the Seventh International Conference on Very Large Data Bases*, pp 144–154, URL <https://web.stanford.edu/class/archive/cs/cs240/cs240.1236/old/sp2014/readings/Gray81.pdf>
13. He J, Yu D (2026) Beyond memory: A transactional continuity kernel for long-lived AI agents. <https://doi.org/10.48550/arXiv.2608.11632>, URL <https://arxiv.org/abs/2608.11632>, preprint, arXiv:2608.11632
14. He J, Yu D (2026) Stored is not supported: Typed provenance and assertion guardrails for persistent AI agents. <https://doi.org/10.48550/arXiv.2609.02127>, URL <https://arxiv.org/abs/2609.02127v1>, preprint, version 1, 2 September 2026, arXiv:2609.02127
15. He J, Veltri E, Santoro D, et al (2016) Interactive and deterministic data cleaning. In: *Proceedings of the 2016 International Conference on Management of Data*, pp 893–907, <https://doi.org/10.1145/2882903.2915242>
16. Jackson D (2002) Alloy: A lightweight object modelling notation. *ACM Transactions on Software Engineering and Methodology* 11(2):256–290. <https://doi.org/10.1145/505145.505149>
17. Kandel S, Paepcke A, Hellerstein J, et al (2011) Wrangler: Interactive visual specification of data transformation scripts. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pp 3363–3372, <https://doi.org/10.1145/1978942.1979444>
18. Kung HT, Robinson JT (1981) On optimistic methods for concurrency control. *ACM Transactions on Database Systems* 6(2):213–226. <https://doi.org/10.1145/319566.319567>, URL <https://www.eecs.harvard.edu/~htk/publication/1981-tods-kung-robinson.pdf>
19. Marinov D, Khurshid S (2001) TestEra: A novel framework for automated testing of java programs. In: *16th IEEE International Conference on Automated Software Engineering*, pp 22–31, <https://doi.org/10.1109/ASE.2001.989787>

20. Mershad K, Malluhi QM, Ouzzani M, et al (2015) Approving updates in collaborative databases. In: 2015 IEEE International Conference on Cloud Engineering, pp 42–47, <https://doi.org/10.1109/ic2e.2015.31>
21. Moreau L, Missier P (2013) PROV-DM: The PROV data model. W3C recommendation, World Wide Web Consortium, URL <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>
22. OWASP Foundation (n.d.) Transaction authorization cheat sheet. Living security practice guidance, URL https://cheatsheetseries.owasp.org/cheatsheets/Transaction_Authorization_Cheat_Sheet.html, undated; accessed 27 September 2026
23. Saidi W (2026) MutMem: Cryptographically authorized mutation in persistent agent memory. <https://doi.org/10.48550/arXiv.2608.02843>, URL <https://arxiv.org/abs/2608.02843>, preprint, arXiv:2608.02843
24. Torlak E, Jackson D (2007) Kodkod: A relational model finder. In: Tools and Algorithms for the Construction and Analysis of Systems, vol 4424. Springer, p 632–647, https://doi.org/10.1007/978-3-540-71209-1_49
25. Yakout M, Elmagarmid AK, Neville J, et al (2011) Guided data repair. Proceedings of the VLDB Endowment 4(5):279–289. <https://doi.org/10.14778/1952376.1952378>
26. Zhou H, Zhang L, Fan Y, et al (2026) LatticeMind: A conflict-aware memory primitive for multi-agent systems. <https://doi.org/10.48550/arXiv.2608.08236>, URL <https://arxiv.org/abs/2608.08236>, preprint, arXiv:2608.08236